Project - Low Level Design

on

Dockerized Construction Python Flask Services

Course Name: DevOps

Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	SNEHA GUPTA	EN22CS306051
2	NAMAN SUNDRIYAL	EN22CS304042
3	SHUBH RATHOD	EN22CS304060
4	AKASH SINGH	EN22CS303009
5	HARSH NANDWAL	EN22CS304027
6	YASH DHIMOLE	EN22CS304066

Group Name: Group 12D11

Project Number: DO-38

Industry Mentor Name: Mr. Vaibhav Sir

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-26

Introduction

1.1 Scope of the Document

This document describes the Low-Level Design of a Dockerized Construction Service built using C# and .NET. The application provides APIs to manage construction reports and is containerized using Docker for consistent deployment.

This document covers:

- Internal system components (Controller, Service, Repository)
- API structure and data flow
- Database design
- Docker and Docker Compose setup
- CI/CD using GitHub Actions

1.2 Intended Audience

This document is intended for:

- Developers working on the application using C# to understand system implementation
- Reviewers to evaluate the design and functionality of the system
- Team members involved in development and maintenance
- Personnel responsible for deployment using Docker and Docker Compose
- Contributors managing CI/CD pipelines using GitHub Actions

This document provides a clear understanding of the system's internal design, workflow, and deployment process.

1.3 System Overview

The system is a containerized Construction Service developed using C# and .NET that provides RESTful APIs to manage construction-related data such as site reports, work details, and issues.

The application is packaged using Docker with a multi-stage build approach to ensure optimized image size and security. It is deployed and managed using Docker Compose, which coordinates the application and database services.

The system follows a layered architecture consisting of Controller, Service, and Repository layers, enabling clear separation of concerns and easy maintainability. It also integrates CI/CD pipelines using GitHub Actions for automated build and deployment.

Overall, the system ensures a scalable, portable, and consistent environment for development and execution.

2. System Design

2.1 Application Design

The application is developed using ASP.NET Core Web API based on a layered (N-tier) architecture to ensure modularity, scalability, and maintainability.

Layers:

- Controller Layer:
  Handles HTTP requests, performs input validation, and sends responses.
- Service Layer:
  Contains core business logic such as processing construction data and generating reports.
- Repository Layer:
  Performs database operations (CRUD) using Entity Framework Core.
- Database Layer:
  Stores structured data like reports, projects, and users.

Deployment Design:

- Application is containerized using Docker
- Multi-container setup managed via Docker Compose
- CI/CD pipeline automated using GitHub Actions

2.2 Process Flow

1. Client sends HTTP request to the API
2. Controller receives and validates the request
3. Request is passed to the Service layer
4. Service processes business logic
5. Repository performs database operations
6. Data is returned back to the Service
7. Controller sends JSON response to the client

2.2 Process Summary

The system follows a structured request-response process where the client request is handled by the Controller, processed by the Service layer, and stored or retrieved using the Repository layer from the database. The processed result is then returned to the client as a JSON response.

The application runs in a containerized environment using Docker and is managed with Docker Compose, ensuring consistency and scalability.

2.3 Information Flow

The information flow in the system defines how data moves between different components of the application during request processing.

Flow Description:

The information flow describes how data moves within the system:

- Input data is received from the client through API requests
- Data is processed in the Controller and Service layers
- The Repository layer sends and retrieves data from the database
- Processed data is returned to the client as a JSON response

The system runs in a containerized environment using Docker, ensuring smooth and consistent data handling.

2.4 Internal Flow

The internal flow represents how data moves within the application components:

- The Controller receives the request and forwards it to the Service layer
- The Service layer processes the logic and calls the Repository
- The Repository interacts with the database for data operations
- The result is passed back from Repository → Service → Controller

The entire flow runs inside a containerized environment using Docker.

2.5 Output Flow

The output flow describes how the processed result is returned to the client:

- Data is retrieved from the database by the Repository
- The Service layer processes and formats the data
- The Controller prepares the final response
- The response is sent to the client in JSON format

The response is generated from the containerized application running using Docker.

2.4 Components Design

1. Controller Component

- Handles incoming HTTP requests
- Acts as entry point of the application
- Validates request data
- Sends response to client

2. Service Component

- Contains business logic
- Processes data from Controller
- Applies validations
- Communicates with Repository

3. Repository Component

- Interacts with database
- Performs CRUD operations

- Retrieves and stores data
- Separates database logic

4. Container Component

- Runs application using Docker
- Ensures consistent environment
- Supports application deployment
- Isolates application from system

5. Containerization Module

- Packages application using Docker
- Uses multi-stage build for optimized image size
- Ensures portability across environments

6. Orchestration Module

- Manages multi-container setup using Docker Compose
- Handles networking between app and database
- Manages environment variables

7. CI/CD Module

- Automates build, test, and deployment using GitHub Actions
- Ensures continuous integration and delivery

2.6 API Catalogue

- GET /reports → Get all reports
- GET /reports/{id} → Get report by ID
- POST /reports → Create report
- PUT /reports/{id} → Update report
- DELETE /reports/{id} → Delete report

External Service APIs

- GitHub API → Used for source code management and repository integration
- GitHub Actions → Automates build, test, and deployment pipeline
- Docker Hub / Container Registry → Stores and retrieves Docker images
- Database Service (SQL Server) → External service for data storage

1. References

- Docker Official Documentation
- Docker Compose Documentation
- ASP.NET Core Documentation
- Entity Framework Core Documentation
- GitHub & GitHub Actions Documentation
- REST API Design Guidelines